

1- Reset by subtract,

- a. Added soft-reset with recurrent
 - i. for this I used Simone's script
- b. Added soft-reset without the recurrent,
 - i. for this I trained the model by myself,
 - ii. Implementation (subtract at the next step)

```
// Vectorized membrane potential update
// Update membrane for soft reset:
//  $v(t+1) = \text{decay} * v(t) + \text{weighted\_inputs} -$ 
reset_value(previous_step)
q15_t temp1[num_neurons], temp2[num_neurons];

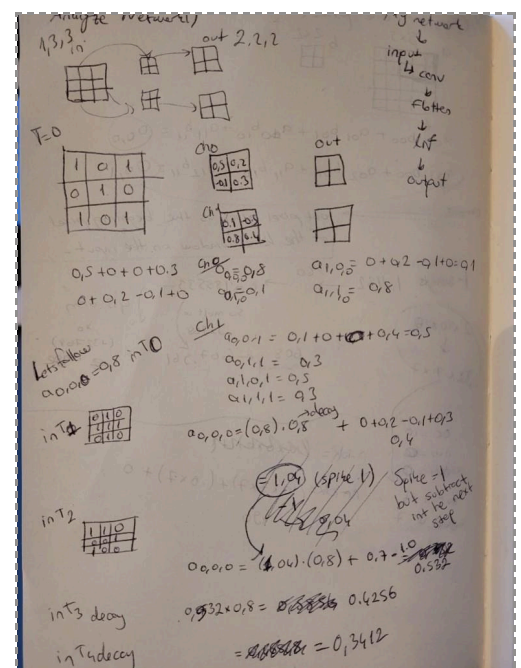
arm_mult_q15(membrane_potentials, decay_factors, temp1,
num_neurons);
arm_add_q15(temp1, weighted_inputs, temp2, num_neurons);
arm_sub_q15(temp2, reset_values, membrane_potentials,
num_neurons);

// Spike check, then store reset_value for the NEXT step
for (uint16_t i = 0; i < num_neurons; i++) {
    if (membrane_potentials[i] > thresholds[i]) {
        output_spikes[i] = 1;
        neurons[i].reset_value = thresholds[i];    // subtract
next step
    } else {
        output_spikes[i] = 0;
        neurons[i].reset_value = 0;
    }
    neurons[i].membrane_potential = membrane_potentials[i];
}
```

- iii. (Metadata need to be added, this way its decided, if its soft or hard reset.
- iv. Maybe I am lacking a decent comparison script to show as a graph like in the SCNN

2-Convolution support

- Trained the model by following the ST-MNIST tutorial in the sntorch tutorial.
- Studied the conv mechanism
- Basic implementation is done, (calculations goes from output to input, thus no sparsity)
- Basic implementation:



```

// 1. Iterate over every output "pixel" (which is one LIF neuron)
for (uint16_t oc = 0; oc < out_ch; oc++) {
    for (uint16_t oh = 0; oh < out_h; oh++) {
        for (uint16_t ow = 0; ow < out_w; ow++) {

            // Accumulator for the current (this is the weighted
input)
            q31_t acc = 0;

            // 2. Perform the Convolution (Sliding Window)
            for (uint16_t ic = 0; ic < in_ch; ic++) {
                for (uint16_t fy = 0; fy < kh; fy++) {
                    for (uint16_t fx = 0; fx < kw; fx++) {

                        // Calculate input coordinates
                        int16_t ih = oh * stride + fy - padding;
                        int16_t iw = ow * stride + fx - padding;

                        // Check boundaries (Padding logic)
                        if (ih >= 0 && ih < in_h && iw >= 0 && iw <
in_w) {

                            // Indexing for [CH][H][W] format
                            uint32_t input_idx = (ic * in_h * in_w)
+ (ih * in_w) + iw;

                            // Indexing for [OutCH][InCH][KH][KW]
format
                            uint32_t weight_idx = (oc * in_ch * kh *
kw) + (ic * kh * kw) + (fy * kw) + fx;

                            acc += (q31_t)input_spikes[input_idx] *
weights[weight_idx];

                        }
                    }
                }
            }
        }
    }
}

```

```

        // 3. LIF Neuron Update Logic
        // Index of the specific neuron in the flat array
        uint32_t n_idx = (oc * out_h * out_w) + (oh * out_w) +
ow;

        // Pull parameters into q31 to avoid premature
saturation
        q31_t v_prev =
(q31_t)neurons[n_idx].membrane_potential;
        q31_t reset = (q31_t)neurons[n_idx].reset_value; //
I will use this reset to achieve soft-reset subtraction in the next
timestep.

        q31_t decay = (q31_t)neurons[n_idx].decay_factor;
        q31_t threshold = (q31_t)neurons[n_idx].threshold;

        // All arithmetic stays in q31 - acc is already in Q15
scale (spike * Q15_weight)
        q31_t v_shifted = (v_prev * decay) >> 15;
        q31_t v_new = v_shifted + acc - reset; // acc added
here before any saturation

        // Reset value consist the threshold from the last
timestep if there was any spike, otherwise its zero. (Uth*S(t)).
        // v_new = ((v_prev)*decay) + acc - reset
        //  $U(t+1) = (U(t) * \text{Beta}) + W * X(t+1) - U_{th} * S(t)$ 
        // This is the equation from the snntorch tutorial 3.

        // Only saturate when writing back to the q15_t struct
field
        neurons[n_idx].membrane_potential = (q15_t)__SSAT(v_new,
16);

        // SOFT RESET
        if (neurons[n_idx].membrane_potential >
neurons[n_idx].threshold) {
            output_spikes[n_idx] = 1;

```

```

        neurons[n_idx].reset_value    = (q15_t)threshold;
// will subtract next step
    } else {
        output_spikes[n_idx] = 0;
        neurons[n_idx].reset_value    = 0;
// clear if there is no spike
    }

```

e. Tested, with debug prints

```

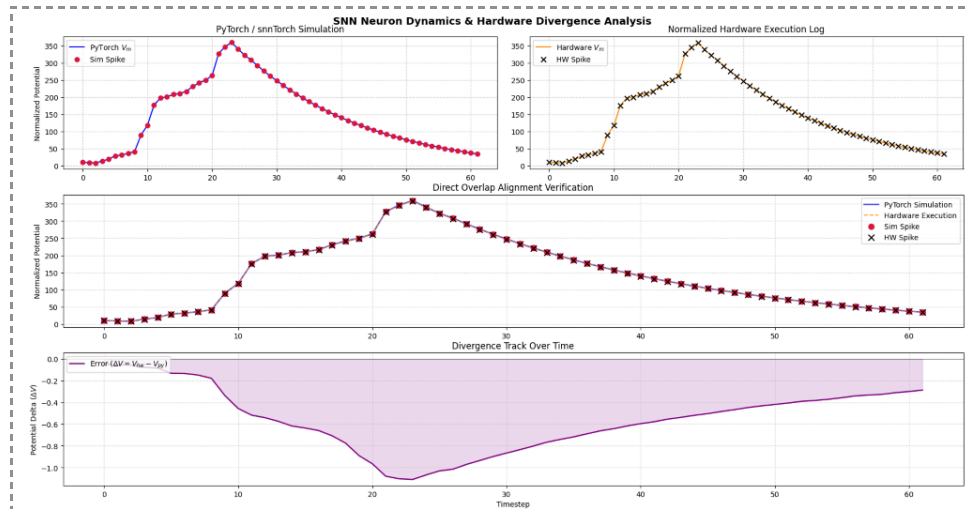
        // I will watch the membrane potential of the firts
layer's this neuron.
        //if (n_idx == 490 && oc == 10 && oh == 0 && ow == 0 ) {

        // For Layer 2 most spiking one is C14, H1, W2 which
makes the n_idx = (14×3×3)+(1×3)+2 = 131
        // if (n_idx == 131 && oc == 14 && oh == 1 && ow == 2 )
{
    //      char buf[200];
    //      // Use %ld for q31_t (long int) to avoid format
warnings
    //      // We print the raw integer. 60 = 1.0 in float
terms.
    //      snprintf(buf, sizeof(buf), "V:%ld = Reset:%ld +
v_shifted:%ld + acc: %ld| threshold: %d S:%d | nindex = %ld | v_prev =
%ld | decay = %ld \\r\\n",
        //                      (long)neurons[n_idx].membrane_potential,
reset, v_shifted, acc, neurons[n_idx].threshold,
        //                      output_spikes[n_idx], n_idx, v_prev,
decay);

    //      usart1_print(buf);
    // }

```

- f. With this debug and in a similar fashion, also pytorch scripts edited to save the potentials across timestep. Made a comparison and plotted. Here is the result:



- g. This done for three different neuron from different layers, to show the correctness.
- h. This done for three different neuron from different layers, to show the correctness.

- i. At the end the a big test is done:

[147] Tr 70 GT=4 Pred=4 PASS spikes: 2 0 5 2 0 7 23 1 2 11 12	Sample 147 Target: 4 Pred: 4 ✓ Spike Counts: [2, 0, 5, 2, 0, 7, 25, 0, 2, 11, 14]
[148] Tr 40 GT=0 Pred=3 FAIL spikes: 4 0 0 21 0 23 0 0 4 12	Sample 148 Target: 9 Pred: 3 ✗ Spike Counts: [5, 0, 0, 26, 0, 24, 0, 0, 4, 11]
[149] Tr 28 GT=9 Pred=9 PASS spikes: 7 3 2 12 0 7 13 10 20 21	Sample 149 Target: 9 Pred: 9 ✓ Spike Counts: [9, 5, 3, 18, 1, 0, 14, 11, 17, 21]
[150] Tr 52 GT=9 Pred=3 FAIL spikes: 19 4 0 30 0 5 0 5 0 20	Sample 150 Target: 9 Pred: 3 ✗ Spike Counts: [21, 4, 0, 52, 0, 0, 0, 3, 20]
[151] Tr 18 GT=3 Pred=3 PASS spikes: 0 0 0 10 0 3 7 0 1 1	Sample 151 Target: 3 Pred: 3 ✓ Spike Counts: [0, 0, 18, 0, 2, 7, 10, 1, 1]
[152] Tr 47 GT=3 Pred=8 FAIL spikes: 33 0 6 33 2 11 0 17 4 5	Sample 152 Target: 3 Pred: 3 ✗ Spike Counts: [33, 0, 5, 34, 1, 13, 0, 26, 2, 3]
[153] Tr 50 GT=0 Pred=0 PASS spikes: 3 0 0 3 4 18 23 8 11 14	Sample 153 Target: 6 Pred: 6 ✓ Spike Counts: [2, 0, 0, 3, 3, 19, 25, 7, 11, 14]
[154] Tr 51 GT=9 Pred=9 PASS spikes: 0 0 0 20 0 19 0 0 15 37	Sample 154 Target: 9 Pred: 9 ✓ Spike Counts: [0, 0, 29, 0, 22, 0, 0, 15, 35]
[155] Tr 57 GT=2 Pred=5 PASS spikes: 10 0 39 6 0 21 2 20 6 5	Sample 155 Target: 2 Pred: 2 ✓ Spike Counts: [10, 0, 39, 6, 0, 21, 1, 19, 6, 5]
[156] Tr 20 GT=0 Pred=0 PASS spikes: 20 2 10 17 0 20 0 0 0 3	Sample 156 Target: 0 Pred: 0 ✓ Spike Counts: [20, 1, 9, 18, 0, 20, 0, 0, 0, 3]
[157] Tr 47 GT=4 Pred=4 PASS spikes: 8 2 14 0 20 1 1 14 4 4	Sample 157 Target: 4 Pred: 4 ✓ Spike Counts: [9, 1, 15, 6, 31, 0, 2, 14, 6, 4]
[158] Tr 57 GT=0 Pred=8 FAIL spikes: 29 11 0 12 2 10 32 0 6 4	Sample 158 Target: 0 Pred: 0 ✓ Spike Counts: [35, 11, 0, 16, 3, 9, 33, 0, 7, 4]
[159] Tr 88 GT=8 Pred=3 FAIL spikes: 26 17 7 53 10 13 14 17 48 15	Sample 159 Target: 8 Pred: 9 ✗ Spike Counts: [29, 18, 7, 54, 10, 12, 16, 17, 47, 19]
[160] Tr 42 GT=2 Pred=2 PASS spikes: 1 0 37 20 0 1 3 14 7 6	Sample 160 Target: 2 Pred: 2 ✓ Spike Counts: [1, 0, 36, 21, 6, 1, 4, 14, 7, 7]
[161] Tr 58 GT=9 Pred=9 PASS spikes: 19 1 39 9 0 24 14 4 8 34	Sample 161 Target: 9 Pred: 2 ✗ Spike Counts: [28, 1, 39, 9, 0, 25, 14, 3, 9, 35]
[162] Tr 38 GT=7 Pred=7 PASS spikes: 4 0 12 28 0 4 0 38 10 0	Sample 162 Target: 7 Pred: 7 ✓ Spike Counts: [4, 0, 16, 30, 0, 0, 0, 38, 10, 0]
[163] Tr 40 GT=5 Pred=5 FAIL spikes: 33 1 3 47 0 43 0 4 5 19	Sample 163 Target: 5 Pred: 3 ✗ Spike Counts: [33, 1, 3, 45, 0, 45, 0, 3, 2, 19]
[164] Tr 40 GT=0 Pred=4 FAIL spikes: 20 0 5 7 0 20 3 2 1	Sample 164 Target: 6 Pred: 0 ✗ Spike Counts: [1, 0, 23, 6, 2, 45, 0, 0, 0, 20]
[165] Tr 38 GT=9 Pred=5 FAIL spikes: 2 0 23 5 2 24 0 6 0 19	Sample 165 Target: 2 Pred: 2 ✓ Spike Counts: [9, 0, 31, 17, 1, 2, 0, 17, 1, 0]
[166] Tr 32 GT=2 Pred=2 PASS spikes: 10 0 32 17 1 9 0 17 1 0	Sample 166 Target: 4 Pred: 4 ✓ Spike Counts: [6, 7, 8, 42, 59, 0, 4, 8, 6, 0]
[167] Tr 79 GT=4 Pred=4 PASS spikes: 0 6 7 40 60 7 6 9 5 5	Sample 167 Target: 9 Pred: 5 ✗ Spike Counts: [0, 0, 1, 17, 2, 24, 1, 4, 3, 12]
[168] Tr 28 GT=9 Pred=5 FAIL spikes: 5 0 1 18 1 23 1 4 3 7	Sample 168 Target: 4 Pred: 4 ✓ Spike Counts: [17, 15, 25, 2, 18, 29, 0, 1, 26, 20]
[169] Tr 99 GT=9 Pred=9 PASS spikes: 17 13 23 1 17 20 60 23 24 17	Sample 169 Target: 3 Pred: 3 ✓ Spike Counts: [17, 8, 11, 39, 8, 0, 0, 6, 2, 0]
[170] Tr 44 GT=3 Pred=3 PASS spikes: 30 9 11 39 0 0 6 2 0	Sample 170 Target: 6 Pred: 0 ✗ Spike Counts: [25, 10, 3, 5, 0, 9, 10, 0, 0, 3]
[171] Tr 37 GT=0 Pred=8 FAIL spikes: 24 9 4 0 8 11 0 0 2	Sample 171 Target: 7 Pred: 7 ✓ Spike Counts: [13, 0, 7, 16, 7, 11, 0, 34, 13, 0]
[172] Tr 34 GT=7 Pred=7 PASS spikes: 12 1 0 15 0 12 0 34 13 0	Sample 172 Target: 0 Pred: 0 ✓ Spike Counts: [65, 3, 12, 17, 24, 19, 30, 9, 16, 7]
[173] Tr 89 GT=0 Pred=8 PASS spikes: 0 7 3 12 15 20 18 33 9 10 8	Sample 173 Target: 6 Pred: 6 ✓ Spike Counts: [14, 12, 8, 13, 16, 21, 14, 15, 19, 17]
[174] Tr 94 GT=1 Pred=1 PASS spikes: 20 64 17 0 23 20 13 15 19 14	Sample 174 Target: 1 Pred: 1 ✓ Spike Counts: [15, 0, 17, 30, 2, 11, 7, 40, 1, 1]
[175] Tr 55 GT=2 Pred=7 FAIL spikes: 10 0 17 30 2 33 11 39 0 1	Sample 175 Target: 5 Pred: 5 ✓ Spike Counts: [10, 7, 4, 11, 17, 84, 22, 12, 5, 6]
[176] Tr 84 GT=5 Pred=5 PASS spikes: 0 6 3 12 19 84 23 12 4 6	Sample 176 Target: 7 Pred: 9 ✗ Spike Counts: [1, 0, 0, 0, 0, 9, 2, 0, 0, 12]
[177] Tr 19 GT=7 Pred=9 FAIL spikes: 3 9 1 0 0 8 2 0 0 12	Sample 177 Target: 6 Pred: 4 ✗ Spike Counts: [4, 0, 2, 2, 14, 6, 2, 3, 2, 0]
[178] Tr 19 GT=0 Pred=1 FAIL spikes: 1 0 2 1 14 4 2 3 2 0	Sample 178 Target: 1 Pred: 1 ✓ Spike Counts: [10, 67, 20, 9, 18, 20, 13, 22, 48, 5]
[179] Tr 74 GT=1 Pred=1 PASS spikes: 10 66 17 8 17 17 12 21 18 5	Sample 179 Target: 8 Pred: 8 ✓ Spike Counts: [1, 5, 12, 17, 0, 54, 11, 10, 78, 11]
[180] Tr 87 GT=8 Pred=8 PASS spikes: 1 7 11 17 1 53 12 9 7 8 12	Sample 180 Target: 4 Pred: 4 ✓ Spike Counts: [7, 14, 22, 29, 53, 28, 2, 12, 0, 5]
[181] Tr 75 GT=4 Pred=5 PASS spikes: 5 12 22 33 54 25 3 0 0 2	Sample 181 Target: 3 Pred: 3 ✓ Spike Counts: [25, 13, 65, 13, 19, 25, 47, 15, 47]
[182] Tr 71 GT=0 Pred=6 PASS spikes: 17 10 6 12 10 22 46 7 10 24	Sample 182 Target: 6 Pred: 6 ✓ Spike Counts: [14, 12, 8, 13, 16, 21, 49, 8, 9, 25]
[183] Tr 50 GT=4 Pred=5 PASS spikes: 2 31 26 39 50 25 21 29 8 27	Sample 183 Target: 4 Pred: 5 ✗ Spike Counts: [3, 8, 15, 31, 30, 34, 11, 1, 8, 0]
[184] Tr 39 GT=4 Pred=4 PASS spikes: 8 7 8 1 11 0 9 1 0 1	Sample 184 Target: 4 Pred: 4 ✓ Spike Counts: [10, 8, 2, 11, 0, 10, 1, 7, 0]
[185] Tr 79 GT=0 Pred=8 PASS spikes: 47 20 3 44 2 10 1 9 15 12	Sample 185 Target: 0 Pred: 0 ✓ Spike Counts: [19, 4, 44, 2, 18, 1, 11, 18, 13]
[186] Tr 71 GT=0 Pred=8 PASS spikes: 7 28 10 28 14 4 47 10 14 0	Sample 186 Target: 6 Pred: 6 ✓ Spike Counts: [8, 29, 10, 30, 16, 4, 48, 11, 14, 0]
[187] Tr 71 GT=0 Pred=8 PASS spikes: 54 4 9 17 0 42 11 17 7 7	Sample 187 Target: 0 Pred: 0 ✓ Spike Counts: [56, 5, 9, 17, 0, 42, 12, 19, 29, 10]
[188] Tr 71 GT=0 Pred=8 PASS spikes: 18 4 9 0 0 1 7 14 0 63	Sample 188 Target: 9 Pred: 9 ✓ Spike Counts: [15, 5, 9, 1, 8, 1, 5, 15, 0, 65]
[189] Tr 71 GT=6 Pred=5 PASS spikes: 30 3 11 22 7 1 51 14 7 21	Sample 189 Target: 6 Pred: 6 ✓ Spike Counts: [31, 3, 13, 24, 6, 1, 49, 16, 8, 10]
[190] Tr 59 GT=5 Pred=5 PASS spikes: 5 0 2 4 9 47 1 17 7 7	Sample 190 Target: 5 Pred: 5 ✓ Spike Counts: [5, 0, 2, 4, 18, 48, 1, 18, 7, 65]
[191] Tr 59 GT=5 Pred=5 PASS spikes: 24 2 12 0 12 2 0 1 10	Sample 191 Target: 2 Pred: 7 ✗ Spike Counts: [17, 2, 32, 24, 4, 10, 3, 43, 1, 0]
[192] Tr 43 GT=2 Pred=7 FAIL spikes: 15 0 32 23 5 10 3 42 1 0	Sample 192 Target: 4 Pred: 4 ✓ Spike Counts: [3, 30, 25, 40, 57, 27, 20, 27, 7, 31]
[193] Tr 89 GT=4 Pred=8 PASS spikes: 2 31 26 39 50 25 21 29 8 27	Sample 193 Target: 0 Pred: 0 ✓ Spike Counts: [26, 1, 3, 12, 0, 11, 4, 8, 7, 9]
[194] Tr 33 GT=0 Pred=8 PASS spikes: 24 2 12 0 12 2 0 1 10	Sample 194 Target: 8 Pred: 8 ✓ Spike Counts: [28, 0, 6, 3, 3, 24, 0, 4, 4, 34]
[195] Tr 48 GT=9 Pred=9 PASS spikes: 30 0 0 3 4 24 0 4 4 32	Sample 195 Target: 8 Pred: 8 ✓ Spike Counts: [6, 1, 2, 9, 1, 10, 0, 7, 17, 13]
[196] Tr 45 GT=8 Pred=5 PASS spikes: 6 1 2 9 1 14 0 7 7 12	Sample 196 Target: 5 Pred: 5 ✓ Spike Counts: [3, 2, 21, 13, 0, 28, 4, 4, 2, 0]
[197] Tr 34 GT=5 Pred=5 PASS spikes: 3 3 22 13 0 29 4 4 7 0	Sample 197 Target: 0 Pred: 0 ✓ Spike Counts: [21, 1, 8, 8, 0, 14, 0, 7, 4, 16]
[198] Tr 47 GT=0 Pred=8 PASS spikes: 22 1 8 10 0 13 5 7 3 15	Sample 198 Target: 9 Pred: 9 ✓ Spike Counts: [18, 7, 11, 8, 3, 5, 13, 6, 4, 46]
[199] Tr 60 GT=9 Pred=9 PASS spikes: 18 5 12 9 3 0 13 8 2 44	Sample 199 Target: 9 Pred: 9 ✓ Spike Counts: [18, 7, 11, 8, 3, 5, 13, 6, 4, 46]

- j. Accuracy: 147/200 (73.50%) Final Accuracy: 150/200, 75.00%
- k. MCU is 147/200, Python is 150/200
- l. As expected the MCU achieved a worse accuracy.

- m. Scale was 360, (So 60V was able to be seen)
- n. Tried with 60.

1471	T= 70	GT=4	Pred=4	PASS	spikes: 2 0 3 2 68 23 0 2 11 16	Sample 147	Target: 4	Pred: 4	✓	Spike Counts: [2, 0, 5, 2, 67, 25, 0, 2, 11, 16]
1481	T= 40	GT=9	Pred=3	FAIL	spikes: 6 0 0 26 0 25 0 0 2 12	Sample 148	Target: 9	Pred: 3	✗	Spike Counts: [5, 0, 0, 26, 0, 24, 0, 0, 4, 11]
1491	T= 28	GT=9	Pred=9	PASS	spikes: 10 5 3 11 9 14 11 17 28	Sample 149	Target: 9	Pred: 9	✓	Spike Counts: [9, 5, 3, 10, 1, 8, 14, 11, 17, 21]
1501	T= 52	GT=9	Pred=3	FAIL	spikes: 17 4 0 30 0 5 0 0 3 22	Sample 150	Target: 9	Pred: 3	✗	Spike Counts: [21, 4, 0, 32, 0, 0, 0, 0, 3, 20]
1511	T= 18	GT=3	Pred=3	PASS	spikes: 0 0 0 18 0 4 0 8 1 1	Sample 151	Target: 3	Pred: 3	✓	Spike Counts: [0, 0, 0, 18, 0, 2, 7, 10, 1, 1]
1521	T= 47	GT=3	Pred=3	PASS	spikes: 29 0 6 35 7 12 0 18 1 1	Sample 152	Target: 3	Pred: 3	✓	Spike Counts: [33, 0, 5, 34, 1, 13, 0, 20, 2, 5]
1531	T= 50	GT=6	Pred=6	PASS	spikes: 2 0 0 3 3 19 23 7 12 14	Sample 153	Target: 6	Pred: 6	✓	Spike Counts: [2, 0, 0, 3, 3, 19, 25, 7, 11, 14]
1541	T= 51	GT=9	Pred=9	PASS	spikes: 0 0 0 26 0 21 0 0 15 37	Sample 154	Target: 9	Pred: 9	✓	Spike Counts: [0, 0, 0, 25, 0, 22, 0, 0, 15, 35]
1551	T= 57	GT=2	Pred=2	PASS	spikes: 11 0 39 6 0 22 2 19 4 5	Sample 155	Target: 2	Pred: 2	✓	Spike Counts: [10, 0, 39, 6, 0, 21, 1, 19, 6, 4]
1561	T= 20	GT=0	Pred=0	PASS	spikes: 20 1 9 18 0 20 0 0 0 2	Sample 156	Target: 0	Pred: 0	✓	Spike Counts: [20, 1, 9, 18, 0, 20, 0, 0, 0, 2]
1571	T= 47	GT=4	Pred=4	PASS	spikes: 8 2 14 7 32 1 3 14 5 2	Sample 157	Target: 4	Pred: 4	✓	Spike Counts: [9, 1, 15, 6, 31, 0, 2, 14, 6, 4]
1581	T= 57	GT=0	Pred=0	PASS	spikes: 24 13 0 12 2 12 23 0 2 4	Sample 158	Target: 0	Pred: 0	✓	Spike Counts: [35, 11, 0, 16, 3, 9, 33, 0, 7, 4]
1591	T= 88	GT=8	Pred=3	FAIL	spikes: 28 17 0 55 9 13 15 16 46 21	Sample 159	Target: 8	Pred: 3	✗	Spike Counts: [29, 18, 7, 54, 10, 12, 16, 17, 47, 19]
1601	T= 42	GT=2	Pred=2	PASS	spikes: 1 0 36 21 6 1 3 14 7 6	Sample 160	Target: 2	Pred: 2	✓	Spike Counts: [1, 0, 36, 21, 6, 1, 4, 14, 7, 7]
1611	T= 58	GT=9	Pred=2	FAIL	spikes: 13 1 36 10 0 25 14 6 9 36	Sample 161	Target: 9	Pred: 2	✗	Spike Counts: [28, 1, 39, 9, 0, 25, 14, 3, 9, 35]
1621	T= 38	GT=7	Pred=7	PASS	spikes: 4 0 13 30 0 1 0 30 11 6	Sample 162	Target: 7	Pred: 7	✓	Spike Counts: [4, 0, 16, 30, 0, 0, 0, 30, 10, 0]
1631	T= 48	GT=5	Pred=3	FAIL	spikes: 35 1 3 46 0 44 0 4 5 20	Sample 163	Target: 5	Pred: 3	✗	Spike Counts: [35, 1, 3, 45, 0, 45, 0, 3, 2, 19]
1641	T= 40	GT=0	Pred=0	FAIL	spikes: 28 1 3 7 0 1 22 3 2 1	Sample 164	Target: 0	Pred: 0	✗	Spike Counts: [31, 1, 3, 6, 0, 2, 23, 2, 3, 1]
1651	T= 38	GT=9	Pred=2	FAIL	spikes: 2 0 26 6 2 23 0 0 0 19	Sample 165	Target: 9	Pred: 5	✗	Spike Counts: [1, 0, 23, 6, 2, 25, 0, 0, 0, 20]
1661	T= 32	GT=2	Pred=2	PASS	spikes: 10 0 32 17 1 3 0 17 1 0	Sample 166	Target: 2	Pred: 2	✓	Spike Counts: [9, 0, 31, 17, 1, 2, 0, 17, 1, 0]
1671	T= 79	GT=4	Pred=4	PASS	spikes: 6 7 8 41 58 8 6 18 6 5	Sample 167	Target: 4	Pred: 4	✓	Spike Counts: [6, 7, 8, 42, 59, 8, 4, 0, 6, 6]
1681	T= 28	GT=9	Pred=3	FAIL	spikes: 6 0 7 10 2 23 1 4 3 9	Sample 168	Target: 9	Pred: 5	✗	Spike Counts: [6, 0, 3, 17, 2, 24, 1, 4, 3, 12]
1691	T= 99	GT=6	Pred=6	PASS	spikes: 7 7 4 1 10 28 44 26 17 17	Sample 169	Target: 6	Pred: 6	✓	Spike Counts: [17, 15, 25, 2, 18, 39, 43, 26, 26, 20]
1701	T= 44	GT=3	Pred=3	PASS	spikes: 37 10 11 39 8 0 6 7 2 0	Sample 170	Target: 3	Pred: 3	✓	Spike Counts: [37, 8, 11, 39, 8, 0, 0, 7, 2, 0]
1711	T= 37	GT=6	Pred=6	FAIL	spikes: 24 10 4 3 0 7 8 0 0 2	Sample 171	Target: 6	Pred: 0	✗	Spike Counts: [25, 10, 3, 5, 0, 9, 10, 0, 0, 3]
1721	T= 34	GT=7	Pred=7	PASS	spikes: 11 1 8 15 8 12 0 34 13 0	Sample 172	Target: 7	Pred: 7	✓	Spike Counts: [13, 0, 7, 16, 7, 11, 0, 34, 13, 0]
1731	T= 89	GT=0	Pred=0	PASS	spikes: 48 0 12 18 27 21 38 10 16 10	Sample 173	Target: 0	Pred: 0	✓	Spike Counts: [65, 3, 12, 17, 20, 19, 36, 9, 10, 7]
1741	T= 94	GT=1	Pred=1	PASS	spikes: 22 61 17 6 24 21 10 16 21 15	Sample 174	Target: 1	Pred: 1	✓	Spike Counts: [20, 66, 18, 9, 24, 22, 14, 15, 19, 17]
1751	T= 55	GT=2	Pred=7	FAIL	spikes: 15 0 20 33 2 11 8 42 0 1	Sample 175	Target: 2	Pred: 7	✗	Spike Counts: [15, 0, 17, 30, 2, 11, 1, 40, 1, 1]
1761	T= 84	GT=5	Pred=5	PASS	spikes: 9 5 4 12 17 04 23 12 3 9	Sample 176	Target: 5	Pred: 5	✓	Spike Counts: [10, 7, 4, 11, 17, 04, 22, 12, 5, 0]
1771	T= 19	GT=7	Pred=5	FAIL	spikes: 3 9 0 0 0 9 2 0 0 12	Sample 177	Target: 7	Pred: 9	✗	Spike Counts: [3, 9, 0, 0, 0, 9, 2, 0, 0, 12]
1781	T= 19	GT=6	Pred=4	FAIL	spikes: 2 0 2 1 14 5 1 3 2 0	Sample 178	Target: 6	Pred: 4	✗	Spike Counts: [4, 0, 2, 2, 14, 5, 1, 3, 2, 0]
1791	T= 74	GT=1	Pred=1	PASS	spikes: 5 67 13 11 17 13 11 22 8 5	Sample 179	Target: 1	Pred: 1	✓	Spike Counts: [10, 67, 20, 9, 10, 20, 13, 22, 18, 5]
1801	T= 87	GT=8	Pred=3	FAIL	spikes: 2 6 0 10 1 50 11 10 58 12	Sample 180	Target: 8	Pred: 8	✓	Spike Counts: [1, 5, 12, 17, 0, 54, 13, 10, 78, 13]
1811	T= 75	GT=4	Pred=8	PASS	spikes: 8 13 18 28 52 30 0 10 0 4	Sample 181	Target: 4	Pred: 4	✓	Spike Counts: [7, 15, 22, 29, 53, 38, 2, 12, 0, 5]
1821	T= 89	GT=3	Pred=3	PASS	spikes: 25 12 11 64 14 18 27 49 15 46	Sample 182	Target: 3	Pred: 3	✓	Spike Counts: [25, 13, 12, 65, 13, 19, 25, 47, 15, 47]
1831	T= 71	GT=6	Pred=6	PASS	spikes: 16 7 4 12 9 10 40 6 8 27	Sample 183	Target: 6	Pred: 6	✓	Spike Counts: [14, 12, 8, 13, 16, 21, 49, 0, 9, 23]
1841	T= 58	GT=4	Pred=5	FAIL	spikes: 3 7 18 32 30 34 11 1 8 0	Sample 184	Target: 4	Pred: 5	✗	Spike Counts: [3, 8, 15, 31, 30, 36, 11, 1, 8, 0]
1851	T= 39	GT=4	Pred=2	FAIL	spikes: 9 7 11 1 3 0 9 1 4 1	Sample 185	Target: 4	Pred: 4	✓	Spike Counts: [10, 8, 8, 2, 11, 0, 10, 1, 7, 0]
1861	T= 79	GT=0	Pred=0	PASS	spikes: 52 22 5 42 2 14 0 4 15 7	Sample 186	Target: 0	Pred: 0	✓	Spike Counts: [47, 19, 4, 44, 2, 10, 1, 11, 18, 13]
1871	T= 71	GT=6	Pred=6	PASS	spikes: 8 28 10 30 17 2 48 11 13 0	Sample 187	Target: 6	Pred: 6	✓	Spike Counts: [8, 28, 10, 30, 16, 4, 48, 11, 14, 0]
1881	T= 71	GT=0	Pred=0	PASS	spikes: 46 6 11 18 1 42 17 19 13 9	Sample 188	Target: 0	Pred: 0	✓	Spike Counts: [56, 5, 9, 17, 0, 42, 12, 19, 19, 10]
1891	T= 60	GT=9	Pred=9	PASS	spikes: 21 7 10 1 0 1 4 23 6 64	Sample 189	Target: 9	Pred: 9	✓	Spike Counts: [19, 5, 9, 1, 0, 1, 5, 15, 0, 65]
1901	T= 71	GT=6	Pred=6	PASS	spikes: 30 3 11 23 8 1 49 16 8 19	Sample 190	Target: 6	Pred: 6	✓	Spike Counts: [31, 3, 13, 24, 0, 1, 49, 16, 8, 18]
1911	T= 59	GT=5	Pred=5	PASS	spikes: 6 0 7 5 8 48 1 17 7 8	Sample 191	Target: 5	Pred: 5	✓	Spike Counts: [5, 0, 2, 4, 10, 48, 1, 18, 7, 0]
1921	T= 43	GT=2	Pred=7	FAIL	spikes: 16 0 31 24 5 9 3 41 1 0	Sample 192	Target: 2	Pred: 7	✗	Spike Counts: [17, 2, 32, 24, 4, 10, 3, 43, 1, 0]
1931	T= 89	GT=4	Pred=4	PASS	spikes: 3 31 26 39 57 28 20 8 31	Sample 193	Target: 4	Pred: 4	✓	Spike Counts: [3, 30, 25, 40, 51, 27, 20, 27, 7, 31]
1941	T= 33	GT=0	Pred=0	PASS	spikes: 27 1 3 12 0 11 3 8 7 11	Sample 194	Target: 0	Pred: 0	✓	Spike Counts: [26, 1, 3, 12, 0, 11, 4, 8, 7, 9]
1951	T= 48	GT=9	Pred=9	FAIL	spikes: 31 0 0 2 3 25 0 4 4 30	Sample 195	Target: 9	Pred: 9	✓	Spike Counts: [38, 0, 0, 4, 3, 24, 0, 4, 4, 34]
1961	T= 45	GT=8	Pred=8	PASS	spikes: 7 1 2 9 1 15 0 7 19 13	Sample 196	Target: 8	Pred: 8	✓	Spike Counts: [6, 1, 2, 9, 1, 16, 0, 7, 17, 13]
1971	T= 34	GT=5	Pred=5	PASS	spikes: 3 3 21 12 0 28 4 4 2 0	Sample 197	Target: 5	Pred: 5	✓	Spike Counts: [3, 2, 21, 13, 0, 28, 4, 4, 2, 0]
1981	T= 47	GT=0	Pred=0	FAIL	spikes: 15 1 9 0 0 13 0 0 3 16	Sample 198	Target: 0	Pred: 0	✓	Spike Counts: [21, 1, 8, 8, 0, 14, 0, 7, 4, 16]
1991	T= 60	GT=9	Pred=9	PASS	spikes: 10 6 10 12 4 5 14 8 2 47	Sample 199	Target: 9	Pred: 9	✓	Spike Counts: [18, 7, 11, 8, 3, 5, 13, 6, 4, 46]

- o. Accuracy: 144/200 (72.00%) Final Accuracy: 150/200, 75.00%
 - p. MCU is 144/200, Python is 150/200
 - q. So the scaling with 360 is better, thus the saturation is more important than the resolution for the model that I have. Make sense bcs soft-reset residue matters
- ### 3- MCU deployment.
- a. First CMSIS-NN and CMSIS-DPS libraries added to the project.
 - b. UART DMA, needed for a big dataset while testing Simone's tests bcs of the difference in the FLASH size.
 - i. UART-DMA has been setup, and python script created to send the test data.

```

Sample 125/140: Pred=5, Exp=5 [✓ Match]
Handshake done

Sample 126/140: Pred=4, Exp=4 [✓ Match]
Handshake done

Sample 127/140: Pred=5, Exp=5 [✓ Match]
Handshake done

Sample 128/140: Pred=4, Exp=4 [✓ Match]
Handshake done

Sample 129/140: Pred=1, Exp=1 [✓ Match]
Handshake done

Sample 130/140: Pred=6, Exp=6 [✓ Match]
Handshake done

Sample 131/140: Pred=3, Exp=3 [✓ Match]
Handshake done

Sample 132/140: Pred=0, Exp=0 [✓ Match]
Handshake done

Sample 133/140: Pred=3, Exp=3 [✓ Match]
Handshake done

Sample 134/140: Pred=6, Exp=6 [✓ Match]
Handshake done

Sample 135/140: Pred=3, Exp=3 [✓ Match]
Handshake done

Sample 136/140: Pred=1, Exp=1 [✓ Match]
Handshake done

Sample 137/140: Pred=6, Exp=6 [✓ Match]
Handshake done

Sample 138/140: Pred=4, Exp=4 [✓ Match]
Handshake done

Sample 139/140: Pred=2, Exp=3 [✗ Miss]
Handshake done

Sample 140/140: Pred=1, Exp=1 [✓ Match]
-----
Accuracy : 85.00% (119/140)
Duration : 50.30 seconds

```

ii.

c. Quad-SPI flash,

- i. When started with the spiking CNN, the flash was not even enough for the weights.
- ii. So QSPI set up to store the weights and the test data.

4- Further plans,

- a. Will optimize the convolution, planning to go from the spiked input matrix to output matrix, the math will change but the non-spiked input neurons wont be calculated. Currently even though the input neurons are not spiking, they are being calculated. Bcs of the out->in

- i. <https://arxiv.org/pdf/2402.06210>
- ii. This work explains what I will implement in Cortex-m7, theirs is FPGA.

Algorithm 1 Even-based Convolution Loop Flow

```

1: for each output channel do
2:   for each time step do
3:     for each input channel do
4:       for each spike event do
5:         for each kernel column do
6:           for each kernel row do
7:             Calculate affected neuron index
8:             Update neuron membrane potential
              with the corresponding filter weight
9:       for each neuron's membrane pot. in OFM do
10:        Add filter bias to it
11:        Perform comparison & thresholding on it
12:        Apply leakage to it

```

iii.

iv. This is unoptimized pseudo calc., from their work

<https://arxiv.org/pdf/2402.06210>

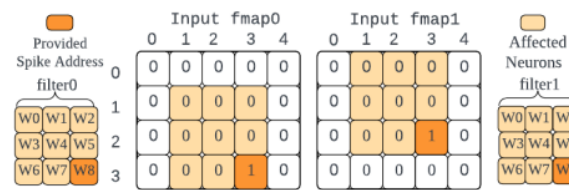


Fig. 1. Illustration of spike-based convolution operation flow. The “membrane potential” of neurons with non-zero activation values is updated along with the surrounding neurons determined by the filter weights.

v.

vi. And this is their input-centric approach (Which I am planning to optimize in this way)

b. “It means that they are still not capable of implementing Convolutional Spiking Neural Networks and focus on Feedforward SNNs so for now they beat all the Python implementations in speed but might not beat the C++ SNNs (there is a lack of benchmarks comparing those C SNNs to existing C++ SNNs - my peer review for ya).”

i. From

https://www.linkedin.com/posts/nw-there-are-snn-in-the-c-programming-language-ugcPost-7470181759704514561-tn_K/?utm_source=social_share_send&utm_medium=android_app&rcm=ACoAABLnSd8BSr1MxeS8bgboP56aKN0XLH47iO4&utm_campaign=copy_link

ii. Professor sent, maybe I can compare my work with C++ version.

c. “CMSIS-NN's `arm_convolve_HWC_q15` is the standard dense ARM convolution. CMSIS-NN achieved 4.6× improvement over a baseline 1D convolution for standard CNNs. Showing that your SNN-specific sparse implementation beats even CMSIS-NN dense for typical SNN firing rates is a strong result.” - Claude

i. <https://arxiv.org/pdf/1801.06601v1>

d. Maybe, I can plot the speedup vs firing rate, when the sparsity implemented, speed-up should increase with the firing rate.

e. I can also measure the efficiency and compare it. “Results on a Xilinx Kintex FPGA (Field Programmable Gate Arrays) using MNIST, FashionMNIST, and SVHN datasets show a 3.14× improvement in accelerator efficiency (FPS/W) compared to a sparsity-oblivious systolic arraybased accelerator. Compared to the most recent sparsity-aware work, our solution improves efficiency by 1.72×” From PULSE(<https://arxiv.org/pdf/2402.06210>)

f.

